Ψηφιακά Συστήματα VLSI

1η Εργαστηριακή Άσκηση

Ομάδα 32: Νικόλαος Λοβέρδος (03120442)

Δημήτριος Μιχαλακόπουλος (03120229)

Βάρδια Γ: 11:30-12:30

Σχολή: Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών

Θέμα Α.2: Δυαδικός αποκωδικοποιητής 3 σε 8

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;

entity dec8 is
    port (
        enc : in  std_logic_vector(2 downto 0);
        dec : out std_logic_vector(7 downto 0)
    );
end dec8;

architecture behavioral of dec8 is
begin
    process(enc)
    begin
        case enc is
            when "000" =>
                dec <= "00000001";
            when "001" =>
                dec <= "00000010";
            when "010" =>
                dec <= "00000100";
            when "011" =>
                dec <= "00001000";
            when "100" =>
                dec <= "00010000";
            when "101" =>
                dec <= "00100000";
            when "110" =>
                dec <= "01000000";
            when "111" =>
                dec <= "10000000";
            when others =>
                dec <= (others => '0');
        end case;
    end process;
end behavioral;

architecture dataflow of dec8 is
begin
    dec(0) <= '1' when enc = "000" else '0';
    dec(1) <= '1' when enc = "001" else '0';
    dec(2) <= '1' when enc = "010" else '0';
    dec(3) <= '1' when enc = "011" else '0';
    dec(4) <= '1' when enc = "100" else '0';
    dec(5) <= '1' when enc = "101" else '0';
    dec(6) <= '1' when enc = "110" else '0';
    dec(7) <= '1' when enc = "111" else '0';
end dataflow;
```

Η behavioral αρχιτεκτονική περιγράφει τη συμπεριφορά του αποκωδικοποιητή χρησιμοποιώντας μια διαδικασία, κατά την οποία αντιστοιχεί κάθε είσοδο (enc) σε μια συγκεκριμένη έξοδο (dec) χρησιμοποιώντας την case. Για παράδειγμα, αν η είσοδος enc είναι "000", η έξοδος dec θα είναι "00000001". Τέλος, αν η είσοδος είναι οτιδήποτε άλλο (when others), θέτει στην έξοδο μόνο μηδενικά.

Η dataflow αρχιτεκτονική περιγράφει τη συμπεριφορά του αποκωδικοποιητή χρησιμοποιώντας παράλληλες εντολές. Ορίζει, δηλαδή, απευθείας τιμές στην έξοδο (dec) με βάση την είσοδο (enc) χρησιμοποιώντας conditional assignments. Για παράδειγμα, αν η είσοδος enc είναι "010", μόνο το τρίτο bit (dec(2)) της εξόδου θα οριστεί σε '1', και όλα τα άλλα bits θα είναι '0'.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.ALL;
use IEEE.numeric_std.all;

entity dec8_tb is
end dec8_tb;

architecture tb_arch of dec8_tb is
    component dec8
        Port (
            enc : in  std_logic_vector(2 downto 0);
            dec : out std_logic_vector(7 downto 0)
        );
    end component;

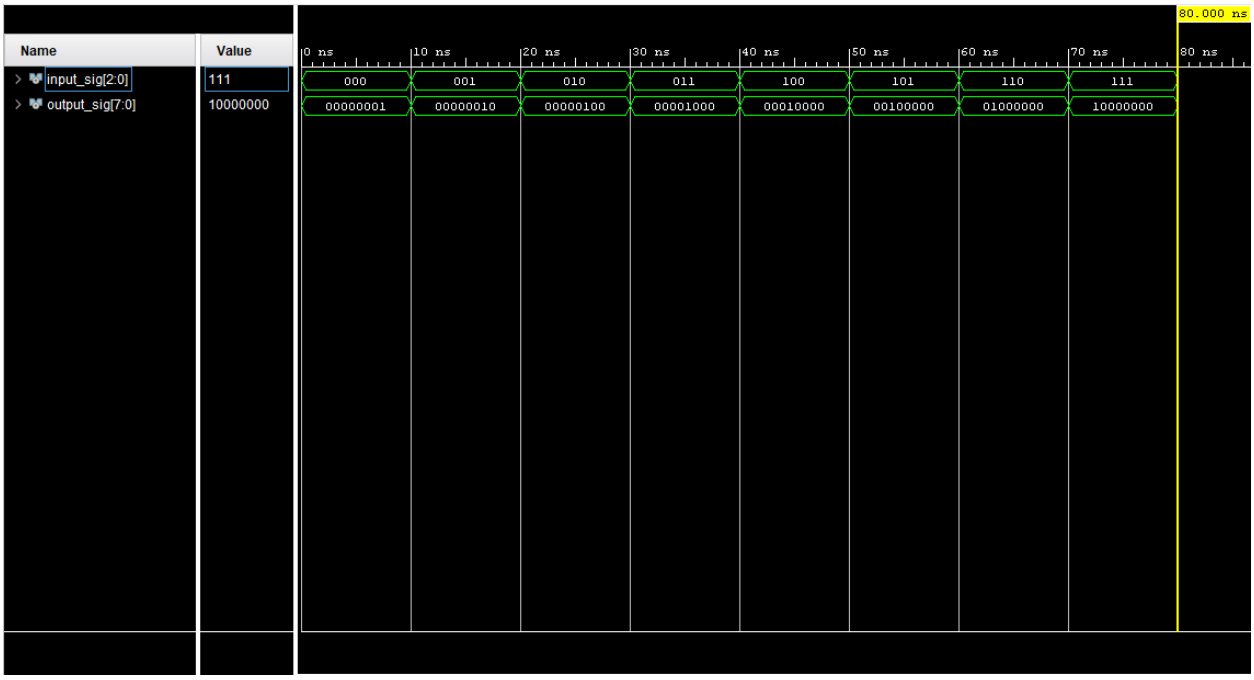
    signal input_sig : std_logic_vector(2 downto 0);
    signal output_sig : std_logic_vector(7 downto 0);

begin
    UUT: dec8 port map (
        enc => input_sig,
        dec => output_sig
    );

    stimulus_process: process
    begin
        for i in 0 to 7 loop
            input_sig <= std_logic_vector(to_unsigned(i, input_sig'length));
            wait for 10 ns;
            assert output_sig = std_logic_vector(to_unsigned(2**i, output_sig'length))
                report "Test case failed: " & integer'image(i)
                severity error;
        end loop;
        wait;
    end process stimulus_process;
end tb_arch;
```

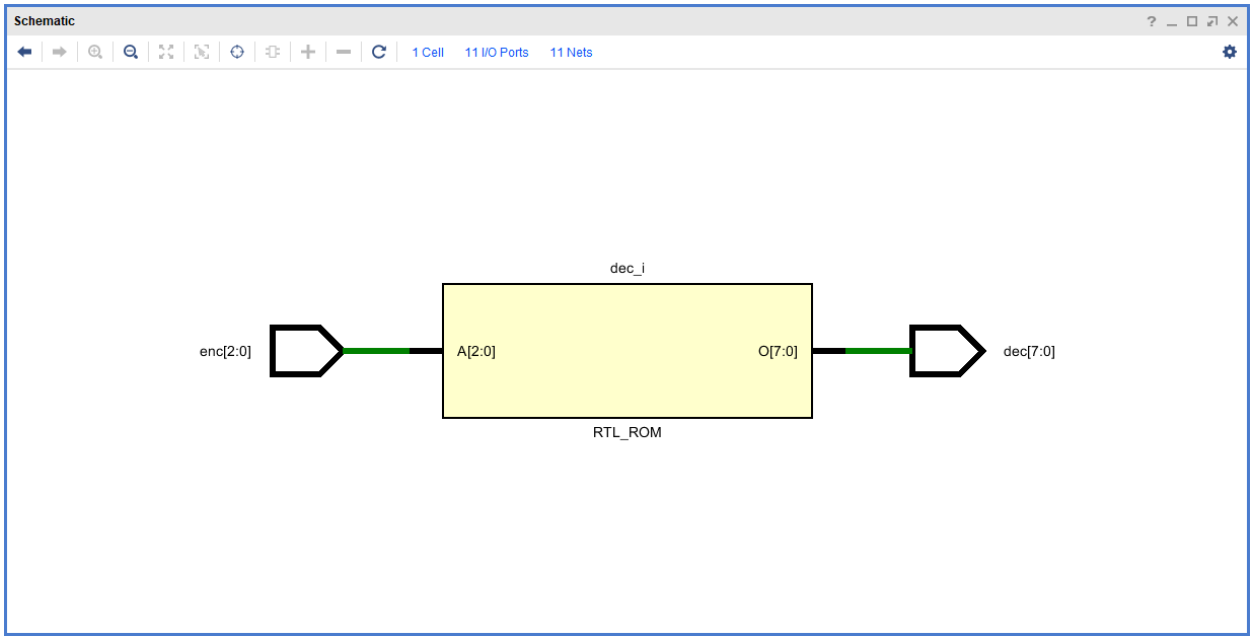
Το παραπάνω testbench εφαρμόζει τις τιμές από 0 έως 7 στην είσοδο του αποκωδικοποιητή και συγκρίνει την έξοδο αυτών σε σχέση με τις αναμενόμενες τιμές εξόδου. Εάν υπάρξει κάποια αναντιστοιχία σε κάποια έξοδο, τότε βγάζει μήνυμα σφάλματος για την αντίστοιχη είσοδο, αλλιώς έχουμε ορθή λειτουργία.

Η προσομοίωση του κυκλώματος, που είναι ίδια και για τις δύο αρχιτεκτονικές, φαίνεται παρακάτω:

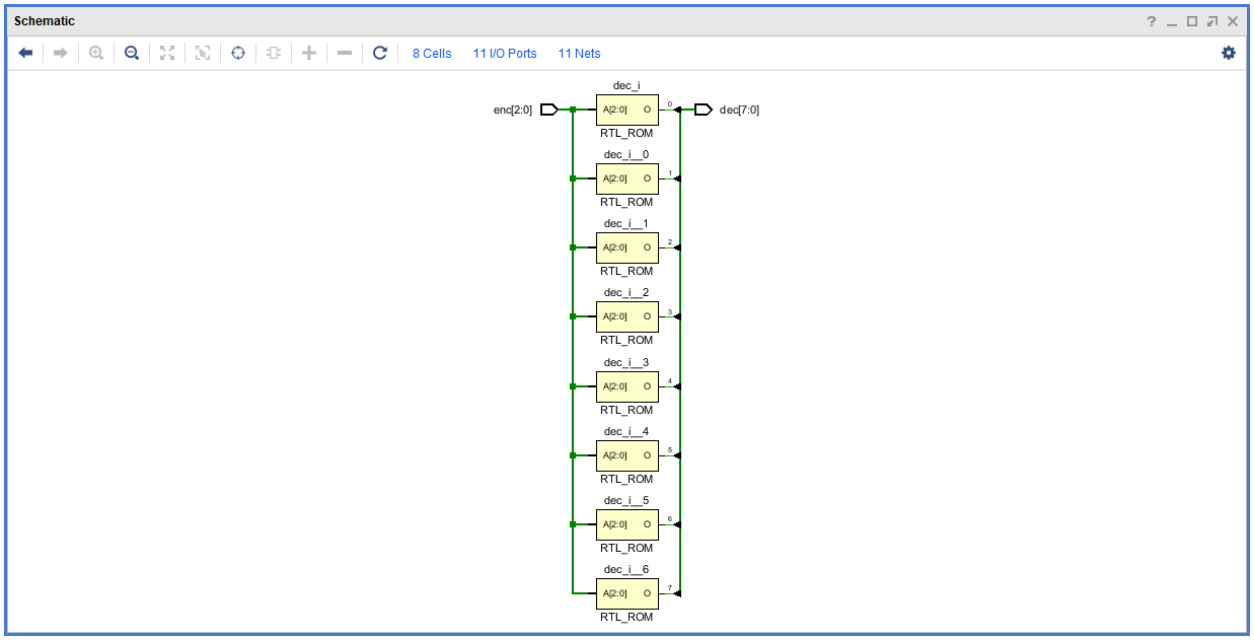


Παρατηρούμε ότι για κάθε τιμή της εισόδου του αποκωδικοποιητή από 0 ως 7, η έξοδος κάθε φορά παίρνει την τιμή που αναμέναμε. Για παράδειγμα, όταν στην είσοδο έχουμε την τιμή "011", περιμένουμε η έξοδος να έχει '1' στο 4^{ο} bit και '0' σε όλα τα υπόλοιπα bits, άρα το "00001000" είναι η σωστή έξοδος. Επιπλέον, δεν εμφανίζεται κάποιο σφάλμα στο Tcl Console, άρα ο δυαδικός αποκωδικοποιητής 3 σε 8 λειτουργεί ορθά.

Το RTL του κυκλώματος για την behavioral αρχιτεκτονική φαίνεται παρακάτω:



Το RTL του κυκλώματος για την dataflow αρχιτεκτονική φαίνεται παρακάτω:



Θέμα Β.2: Καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση

Ο κώδικας που παράγει το ζητούμενο κύκλωμα φαίνεται παρακάτω:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity shift_reg_directional is
5       port (
6           clk, rst, si, en, pl, dir: in std_logic;
7           din: in std_logic_vector(3 downto 0);
8           so: out std_logic
9       );
10  end shift_reg_directional;
11
12  architecture rtl of shift_reg_directional is
13      signal dff: std_logic_vector(3 downto 0);
14      begin
15          edge: process (clk, rst)
16          begin
17              if rst = '0' then
18                  dff <= (others => '0');
19              elsif clk'event and clk = '1' then
20                  if pl = '1' then
21                      dff <= din;
22                  elsif en = '1' then
23                      if dir = '1' then -- Right shift
24                          dff <= si & dff(3 downto 1);
25                          so <= dff(0);
26                      else -- Left shift
27                          dff <= dff(2 downto 0) & si;
28                          so <= dff(3);
29                      end if;
30                  end if;
31              end if;
32      end process;
33  end rtl;
```

Όταν το rst (reset) είναι '0' κάνουμε όλα τα bit του dff '0', δηλαδή μηδενίζουμε τα flip-flops. Έπειτα, σε κάθε θετική ακμή του ρολογιού αν το pl (Parallel Load) είναι '1' τότε περνάμε στο dff το din, δηλαδή περνάμε τα bit της εισόδου στα flip-flops. Εναλλακτικά, αν το en (Enable) είναι '1', τότε αν το dir (Direction) είναι '1' έχουμε δεξιά ολίσθηση, ενώ αν είναι '0' έχουμε αριστερή ολίσθηση. Κατά την δεξιά ολίσθηση οδηγούμε στην έξοδο so (Serial Output) το LSB του dff, μετακινούμε μια θέση δεξιά τα άλλα 3 bit και εισάγουμε στο MSB την είσοδο si (Serial Input).

Αντίστοιχα, κατά την αριστερή ολίσθηση οδηγούμε στην έξοδο so το MSB του dff, μετακινούμε μια θέση αριστερά τα άλλα 3 bit και εισάγουμε στο LSB την είσοδο si.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
1   library IEEE;
2   use IEEE.std_logic_1164.all;
3
4   entity shift_reg_directional_tb is
5   end shift_reg_directional_tb;
6
7   architecture tb_arch of shift_reg_directional_tb is
8       component shift_reg_directional
9           port (
10              clk, rst, si, en, pl, dir : in std_logic;
11              din: in std_logic_vector(3 downto 0);
12              so: out std_logic
13          );
14      end component;
15
16      -- Testbench signals
17      signal clk_tb, rst_tb, si_tb, en_tb, pl_tb, so_tb, dir_tb: std_logic;
18      signal din_tb: std_logic_vector(3 downto 0);
19
20      constant CLK_PERIOD : time := 10 ns; -- Clock period
21
22  begin
23      DUT: shift_reg_directional
24          port map(
25              clk => clk_tb,
26              rst => rst_tb,
27              si => si_tb,
28              en => en_tb,
29              pl => pl_tb,
30              dir => dir_tb,
31              din => din_tb,
32              so => so_tb
33          );
34
35      clk_process: process
36      begin
37          clk_tb <= '0';
38          wait for CLK_PERIOD / 2;
39          clk_tb <= '1';
40          wait for CLK_PERIOD / 2;
41      end process;
42
```

```
    stimulus: process
    begin
        rst_tb <= '1';
        wait for CLK_PERIOD * 2;

        pl_tb <= '1';
        din_tb <= "0010";
        wait for CLK_PERIOD * 5;

        -- Test left shift
        pl_tb <= '0';
        en_tb <= '1';
        dir_tb <= '0';
        si_tb <= '1';
        wait for CLK_PERIOD * 5;

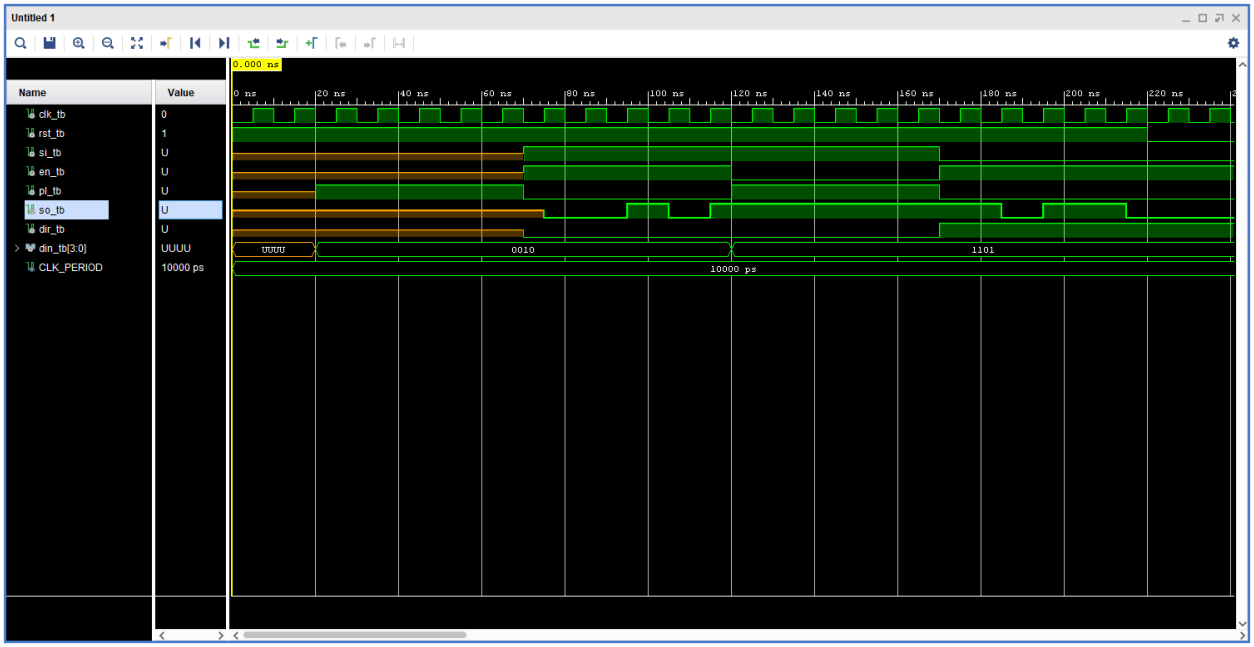
        pl_tb <= '1';
        en_tb <= '0';
        din_tb <= "1101";
        wait for CLK_PERIOD * 5;

        -- Test right shift
        pl_tb <= '0';
        en_tb <= '1';
        dir_tb <= '1';
        si_tb <= '0';
        wait for CLK_PERIOD * 5;

        rst_tb <= '0';
        wait;
    end process;
end tb_arch;
```

Το παραπάνω testbench αρχικά θέτει το reset σε '1' για να μην μηδενίζουν συνέχεια τα flip-flops. Έπειτα πραγματοποιεί παράλληλη φόρτωση του "0010" και στην συνέχεια εκτελεί αριστερή ολίσθηση για 5 κύκλους ρολογιού, έχοντας στην σειριακή είσοδο την τιμή '1'. Μετά κάνει ξανά παράλληλη φόρτωση το "1101" και στην συνέχεια εκτελεί δεξιά ολίσθηση για 5 κύκλους ρολογιού, έχοντας στην σειριακή είσοδο την τιμή '0'. Τέλος, θέτει το reset σε '0' προκειμένου να δοκιμάσουμε και τον ασύγχρονο μηδενισμό του καταχωρητή ολίσθησης.

Η προσομοίωση του κυκλώματος του καταχωρητή φαίνεται παρακάτω:



Η αριστερή ολίσθηση με αφετηρία το "0010" και σειριακή είσοδο '1' γίνεται:

dff:0101 so:0→dff:1011 so:0→dff:0111 so:1→dff:1111 so:0→dff:1111 so:1

άρα στην έξοδο περιμένουμε να δούμε σε 5 κύκλους ρολογιού την τιμή "00101".

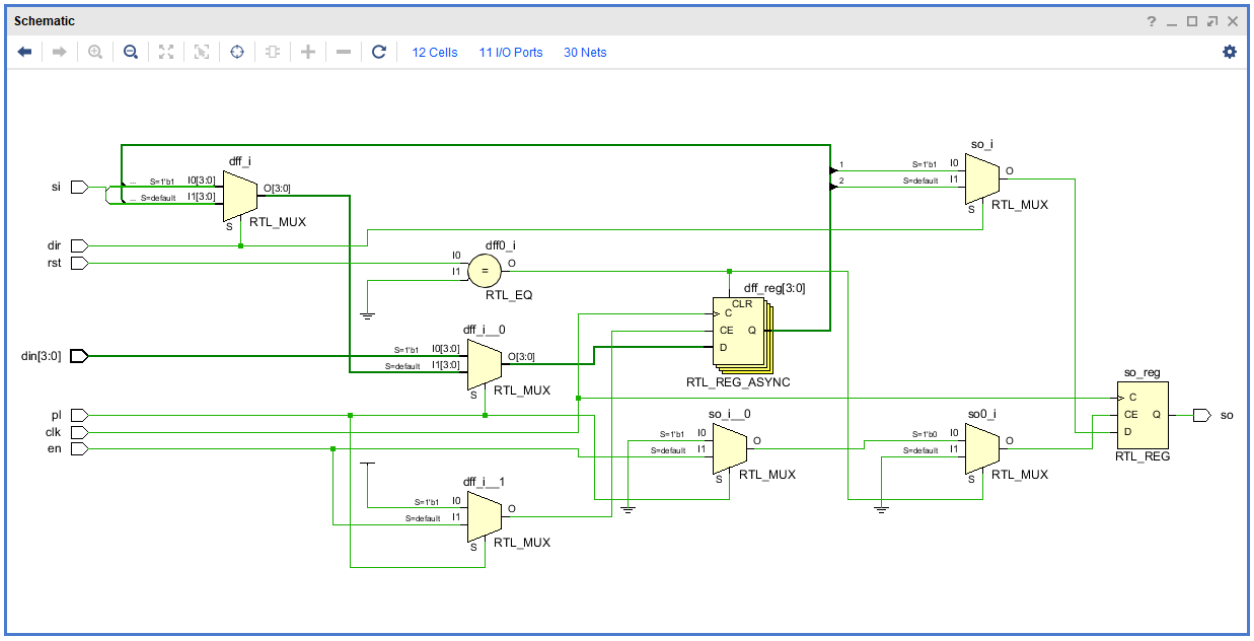
Η δεξιά ολίσθηση με αφετηρία το "1101" και σειριακή είσοδο '0' γίνεται:

dff:0110 so:1→dff:0011 so:0→dff:0001 so:1→dff:0000 so:1→dff:0000 so:0

άρα στην έξοδο περιμένουμε να δούμε σε 5 κύκλους ρολογιού την τιμή "10110".

Παρατηρούμε στην προσομοίωση ότι μετά από 7 κύκλους ρολογιού η σειριακή έξοδος του καταχωρητή παίρνει κατά σειρά τις τιμές "00101" και πάλι μετά από 5 κύκλους ρολογιού η έξοδος παίρνει κατά σειρά τις τιμές "10110". Επομένως, μπορούμε με σιγουριά να πούμε ότι τόσο η δεξιά και η αριστερή ολίσθηση, όσο και η παράλληλη φόρτωση λειτουργούν σωστά. Τέλος, όταν το reset είναι '0' μηδενίζεται επιτυχώς το περιεχόμενο των flip-flops. Συνεπώς, ο καταχωρητής ολίσθησης των 4 bits με παράλληλη φόρτωση λειτουργεί ορθά.

Το RTL του κυκλώματος του καταχωρητή φαίνεται παρακάτω:



Θέμα Β.3: Μετρητής 3 bit με είσοδο ενεργοποίησης και κρατούμενο εξόδου

Ο κώδικας που παράγει το κύκλωμα του up/down μετρητή φαίνεται παρακάτω:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity count3 is
    port(
        clk,
        resetn,
        count_en,
        count_up_en : in std_logic;
        sum         : out std_logic_vector(2 downto 0);
        cout        : out std_logic
    );
end entity count3;

architecture rtl_limit of count3 is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn = '0' then
            count <= (others => '0');
        elsif clk'event and clk='1' then
            if count_en = '1' then
                case count_up_en is
                    when '1' =>
                        if count/=7 then
                            count <= count+1;
                        else
                            count <= (others => '0');
                            cout <= '1';
                        end if;
                    when '0' =>
                        if (count/=0) then
                            count <= count-1;
                        else
                            count <= (others => '1');
                            cout <= '0';
                        end if;
                    when others =>
                        count <= (others => 'X');
                end case;
            end if;
        end if;
    end process;
    sum  <= count;
end architecture rtl_limit;
```

Όταν το resetn είναι '0' τότε κάνουμε όλα τα bit εξόδου '0', δηλαδή ξεκινάει ξανά η μέτρηση από το 0. Έπειτα, σε κάθε θετική ακμή του ρολογιού αν το count_en είναι '1', δηλαδή είναι ενεργή η είσοδος ενεργοποίησης, τότε αρχίζει ο μετρητής να μετράει προς τα πάνω όταν το count_up_en είναι '1' και αντίστοιχα προς τα κάτω όταν το count_up_en είναι '0'. Κατά την μέτρηση προς τα πάνω, όσο η έξοδος είναι διάφορη του 7 αυξάνουμε κατά '1' και όταν φτάσει στο 7, ξεκινάμε ξανά από το 0 και έχουμε κρατούμενο εξόδου '1'. Αντίστοιχα, κατά την μέτρηση προς τα κάτω, όσο η έξοδος είναι διάφορη του 0 μειώνουμε κατά '1' και όταν φτάσει στο 0, ξεκινάμε ξανά από το 7 και έχουμε κρατούμενο εξόδου 0. Σε κάθε άλλη περίπτωση έχουμε στην έξοδο X για να δείξουμε ότι κάπου έγινε σφάλμα.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity count3_tb is
end entity count3_tb;

architecture tb_arch of count3_tb is
    constant CLOCK_PERIOD : time := 10 ns;

    signal clk, resetn, count_en, count_up_en : std_logic := '0';
    signal sum : std_logic_vector(2 downto 0);
    signal cout : std_logic;

    component count3
        port(
            clk, resetn, count_en, count_up_en : in std_logic;
            sum : out std_logic_vector(2 downto 0);
            cout : out std_logic
        );
    end component;

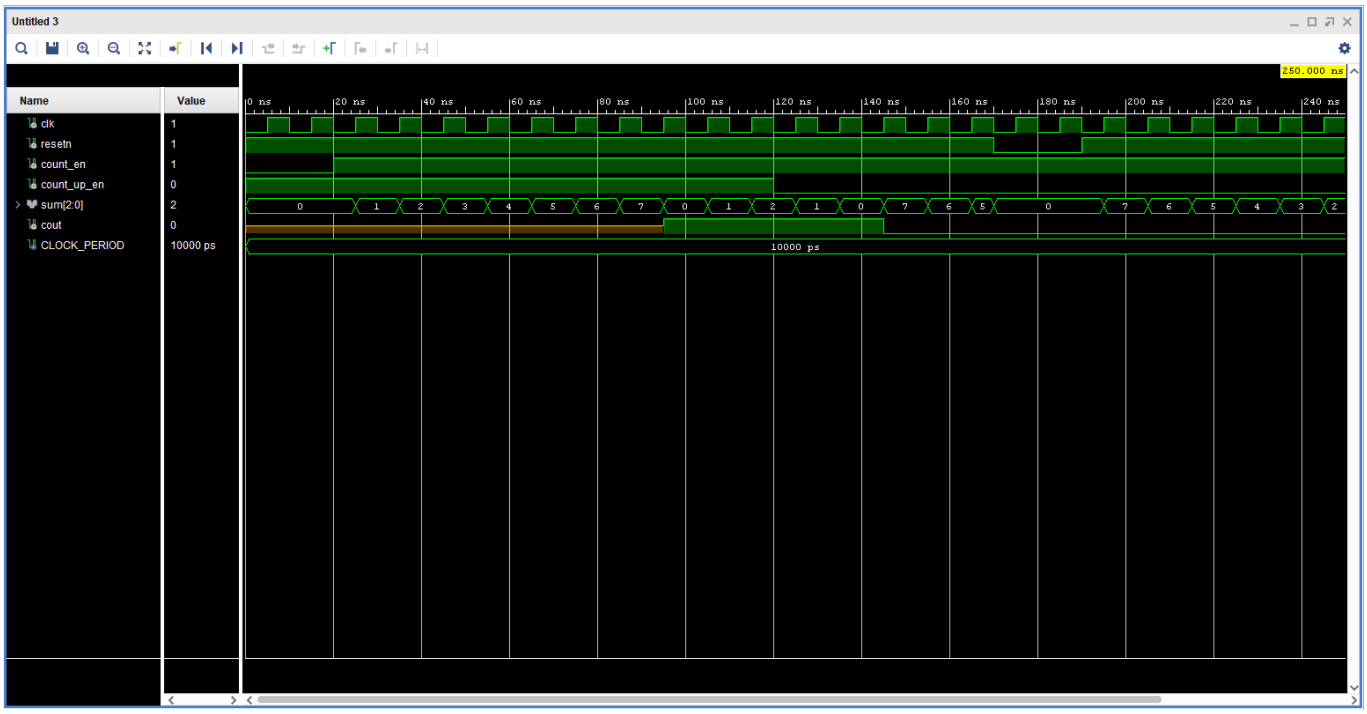
begin
    DUT: count3
    port map (
        clk => clk,
        resetn => resetn,
        count_en => count_en,
        count_up_en => count_up_en,
        sum => sum,
        cout => cout
    );

    clk_process: process
    begin
        while now < 250 ns loop
            clk <= '0';
            wait for CLOCK_PERIOD / 2;
            clk <= '1';
            wait for CLOCK_PERIOD / 2;
        end loop;
        wait;
    end process;
```

```
43
44      stimulus: process
45      begin
46          resetn <= '1';
47          count_en <= '0';
48          count_up_en <= '1';
49          wait for 20 ns;
50
51          count_en <= '1';
52          wait for 100 ns;
53
54          count_up_en <= '0';
55          wait for 50 ns;
56
57          resetn <= '0';
58          wait for 20 ns;
59          resetn <= '1';
60          wait for 50 ns;
61
62          wait;
63      end process;
64
65  end architecture tb_arch;
```

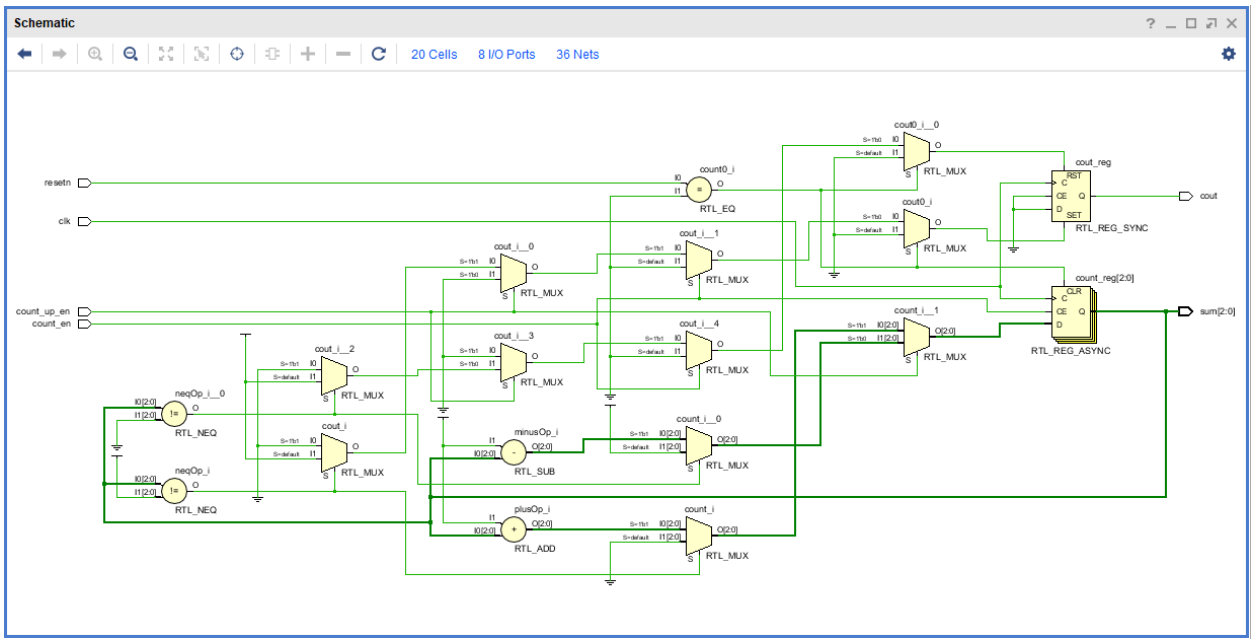
Το παραπάνω testbench ελέγχει αρχικά την είσοδο ενεργοποίησης του μετρητή. Έπειτα, πραγματοποιεί μέτρηση προς τα πάνω για 100ns και στην συνέχεια κάνει μέτρηση προς τα κάτω για 100ns, η οποία όμως διακόπτεται μετά από 20 ns προκειμένου να ελεγχθεί η είσοδος μηδενισμού και έπειτα συνεχίζει η μέτρηση.

Η προσομοίωση του κυκλώματος του up/down μετρητή φαίνεται παρακάτω:



Παρατηρούμε ότι για τα πρώτα 20ns της προσομοίωσης όπου το count_en είναι '0', ο μετρητής μένει σταθερά στο 0, πράγμα που σημαίνει ότι η είσοδος ενεργοποίησης λειτουργεί σωστά. Έπειτα που το count_en γίνεται '1', όσο το count_up_en είναι '1', γίνεται αέναη μέτρηση προς τα πάνω με επαναφορά στο 0 μετά το 7 και κρατούμενο εξόδου '1'. Στην συνέχεια, όσο το count_up_en είναι '0', γίνεται αέναη μέτρηση προς τα κάτω με επαναφορά στο 7 μετά το 0 και κρατούμενο εξόδου '0'. Στο σημείο μάλιστα που γίνεται το resetn '0', βλέπουμε την έξοδο να γίνεται 0, και όταν το resetn ξαναγίνει '1', να συνεχίζει κανονικά η μέτρηση προς τα κάτω. Συνεπώς, ο up/down μετρητής 3 bit λειτουργεί ορθά.

Το RTL του κυκλώματος του up/down μετρητή φαίνεται παρακάτω:



Παρατηρώντας το παραπάνω κύκλωμα βλέπουμε ότι έχει γίνει δύο φορές η ίδια υλοποίηση για την μέτρηση προς τα πάνω και για την μέτρηση προς τα κάτω. Θα μπορούσε αυτή η υλοποίηση να γίνει μια φορά και με έναν πολυπλέκτη να ελέγχουμε αν θέλουμε να κάνουμε άνω ή κάτω μέτρηση αντίστοιχα. Άρα, ναι θα μπορούσε να σχεδιαστεί σε απλούστερη μορφή "με το χέρι".

Ο κώδικας που παράγει το κύκλωμα του μετρητή με παράλληλη είσοδο modulo (όριο μέτρησης) φαίνεται παρακάτω:

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity count3_limit is
    port(
        clk,resetn,count_en : in std_logic;
        target_count : in std_logic_vector(2 downto 0);
        sum : out std_logic_vector(2 downto 0);
        cout : out std_logic
    );
end entity count3_limit;

architecture rtl_limit of count3_limit is
    signal count : std_logic_vector(2 downto 0);
begin
    process(clk, resetn)
    begin
        if resetn = '0' then
            count <= (others => '0');
        elsif clk'event and clk = '1' then
            if count_en = '1' then
                if count/= target_count then
                    count <= count + 1;
                else
                    count <= (others => '0');
                    cout <= '1';
                end if;
            end if;
        end if;
    end process;
    sum <= count;
end rtl_limit;
```

Όταν το resetn είναι '0' τότε κάνουμε όλα τα bit εξόδου '0', δηλαδή ξεκινάει ξανά η μέτρηση από το 0. Έπειτα, σε κάθε θετική ακμή του ρολογιού αν το count_en είναι '1', δηλαδή είναι ενεργή η είσοδος ενεργοποίησης, τότε ο μετρητής αρχίζει να μετράει προς τα πάνω μέχρι να φτάσει το όριο μέτρησης target_count. Όσο η έξοδος είναι διάφορη του ορίου μέτρησης, τότε αυξάνουμε κατά '1' και όταν η μέτρηση φτάσει στο target_count, τότε ξεκινάμε ξανά από το 0 και έχουμε κρατούμενο εξόδου '1'.

Το testbench που εξετάζει την ορθή λειτουργία του κυκλώματος είναι το εξής:

```
library IEEE;
use IEEE.std_logic_1164.all;

entity count3_tb is
end entity count3_tb;

architecture tb_arch of count3_tb is
    constant CLK_PERIOD : time := 10 ns;
    signal clk, resetn, count_en : std_logic := '0';
    signal target_count : std_logic_vector(2 downto 0) := "101";
    signal sum : std_logic_vector(2 downto 0);
    signal cout : std_logic;

    component count3_limit
        port(
            clk, resetn, count_en : in std_logic;
            target_count : in std_logic_vector(2 downto 0);
            sum : out std_logic_vector(2 downto 0);
            cout : out std_logic
        );
    end component;

begin
    DUT: count3_limit
        port map (
            clk => clk,
            resetn => resetn,
            count_en => count_en,
            target_count => target_count,
            sum => sum,
            cout => cout
        );

    clk_process: process
    begin
        while now < 1000 ns loop
            clk <= not clk;
            wait for CLK_PERIOD / 2;
        end loop;
        wait;
    end process clk_process;

    stimulus_process: process
    begin
        resetn <= '1';
        count_en <= '1';
        wait for 50 ns;

        count_en <= '0';
        wait for 50 ns;

        count_en <= '1';
        wait for 50 ns;

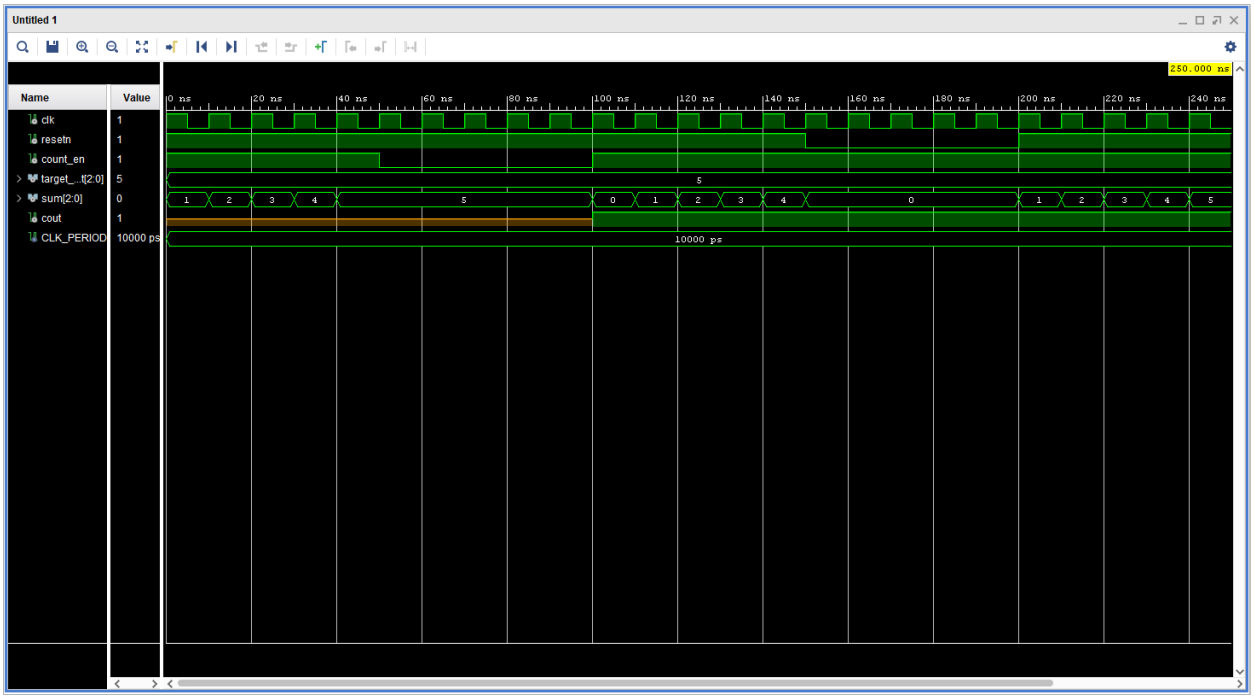
        resetn <= '0';
        wait for 50 ns;

        resetn <= '1';
        wait;
    end process stimulus_process;

end tb_arch;
```

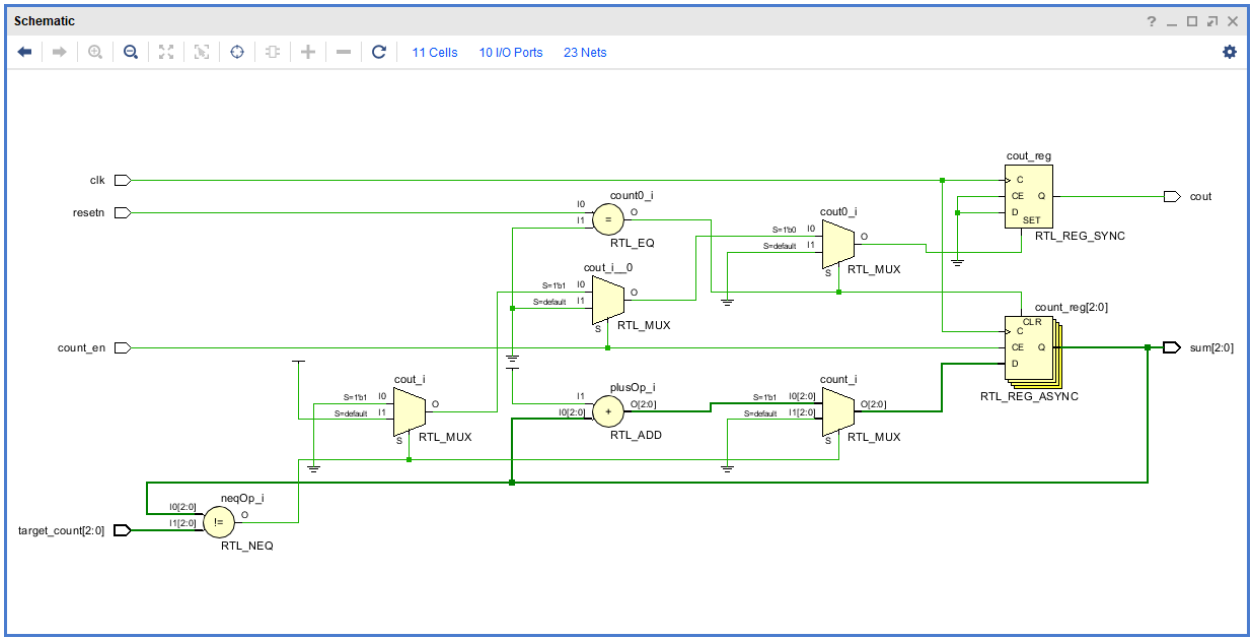
Το παραπάνω testbench έχει ως όριο μέτρησης τον αριθμό 5 ("101"). Αρχικά, ξεκινάει η μέτρηση για 50ns, η οποία διακόπτεται για 50 ns προκειμένου να ελέγξουμε την λειτουργία της εισόδου ενεργοποίησης. Έπειτα, η μέτρηση αρχίζει ξανά, αλλά διακόπτεται πάλι για 50ns προκειμένου να ελέγξουμε την της είσοδο μηδενισμού. Τέλος, η μέτρηση ξαναρχίζει μέχρι το τέλος της προσομοίωσης.

Η προσομοίωση του κυκλώματος του μετρητή με παράλληλη είσοδο modulo (όριο μέτρησης) φαίνεται παρακάτω:



Παρατηρούμε ότι ο μετρητής ξεκινάει κανονικά να μετράει προς τα πάνω μέχρι που το count_en γίνεται '0', όπου και η έξοδος παγώνει, γεγονός που σημαίνει ότι η έισοδος ενεργοποίησης λειτουργεί σωστά. Έπειτα, που το count_en γίνεται ξανά '1', συνεχίζει η μέτρηση μέχρι που το resetn γίνεται '0', όπου η έξοδος του μετρητή γίνεται 0, πράγμα που συμαίνει ότι η είσοδος μηδενισμού λειτουργεί κανονικά. Τέλος, όταν το resetn ξαναγίνεται '1' συνεχίζει κανονικά η μέτρηση μέχρι το τέλος. Ταυτόχρονα, ο μετρητής καθόλη την διάρκεια της προσομοίωσης μετράει σωστά μέχρι τον αριθμό 5 που είναι το όριο μέτρησης και μετά πηγαίνει στο 0 με ταυτόχρονη έξοδο κρατούμενου 1. Συνεπώς, ο μετρητής με παράλληλη είσοδο modulo (όριο μέτρησης) λειτουργεί ορθά.

Το RTL του κυκλώματος του μετρητή με όριο μέτρησης φαίνεται παρακάτω:



Παρατηρώντας το παραπάνω κύκλωμα καταλήγουμε ότι δεν θα μπορούσε να σχεδιαστεί σε απλούστερη μορφή "με το χέρι".